

Deep Learning

Introduction to Deep Learning for Computer Vision



Prof. Anxiao (Andrew) Jiang

Learning Objectives:

1. Understand how a convolutional neural network works.
2. Understand how to train a convolutional neural network using regularization, data augmentation, feature extraction, and fine tuning.



Roadmap of this lecture:

1. How a convolutional neural network (CNN) works

1.1 CNN architecture

1.2 How a convolution layer works

1.3 How max-pooling and flatten layers work

2. Example of CNN for MNIST



Roadmap of this lecture (continued):

3. Train a CNN without regularization
4. Train a CNN with data augmentation
5. Use feature extraction from a trained model
6. Fine-tune a pre-trained model



CNN for Images (or image-like data)

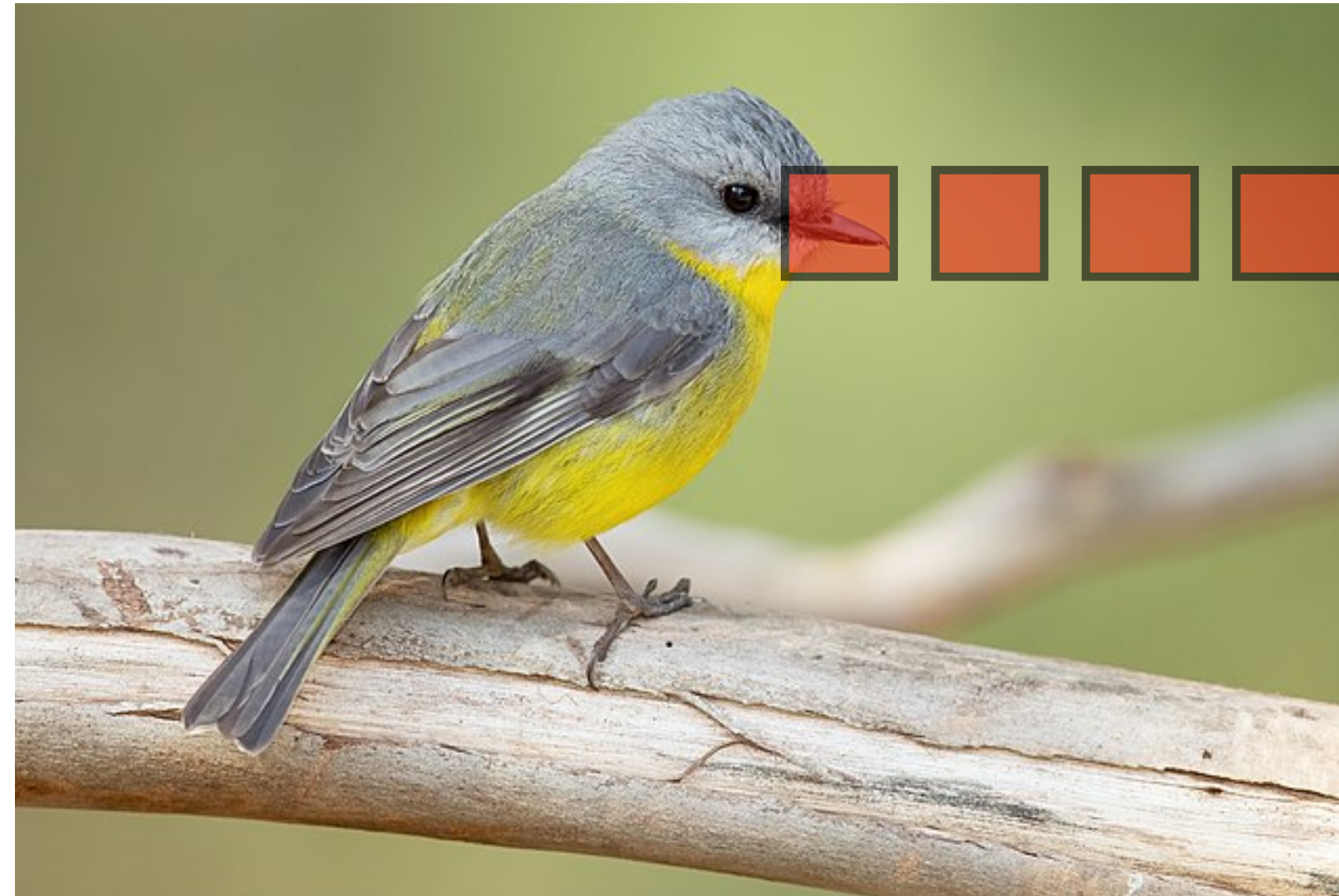


CNN for Images (or image-like data)



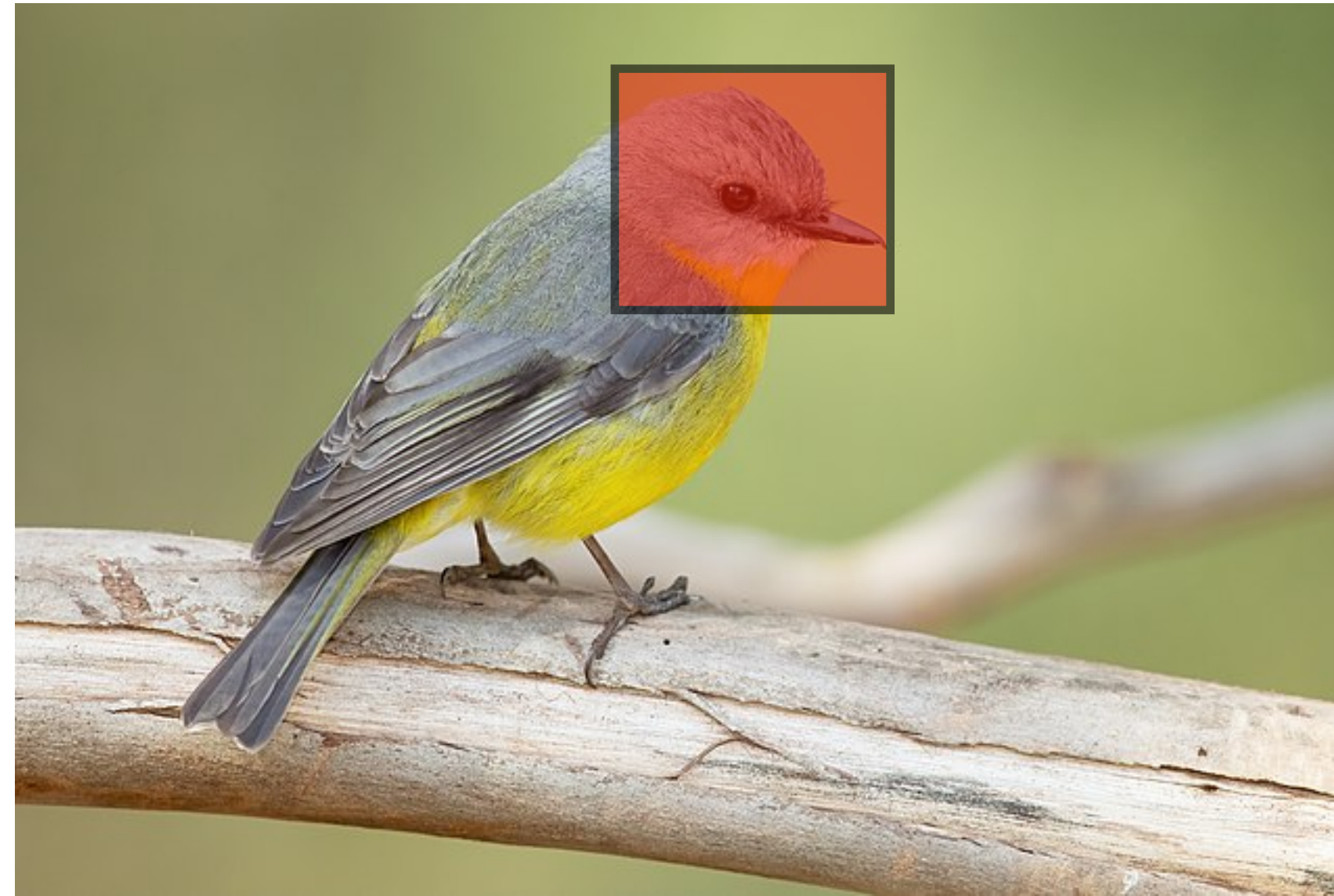
1. Some patterns are much smaller than the image.

CNN for Images (or image-like data)



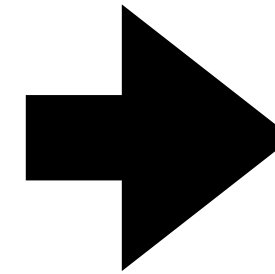
2. We can check different parts of the image to detect a pattern using the same “filter”.

CNN for Images (or image-like data)



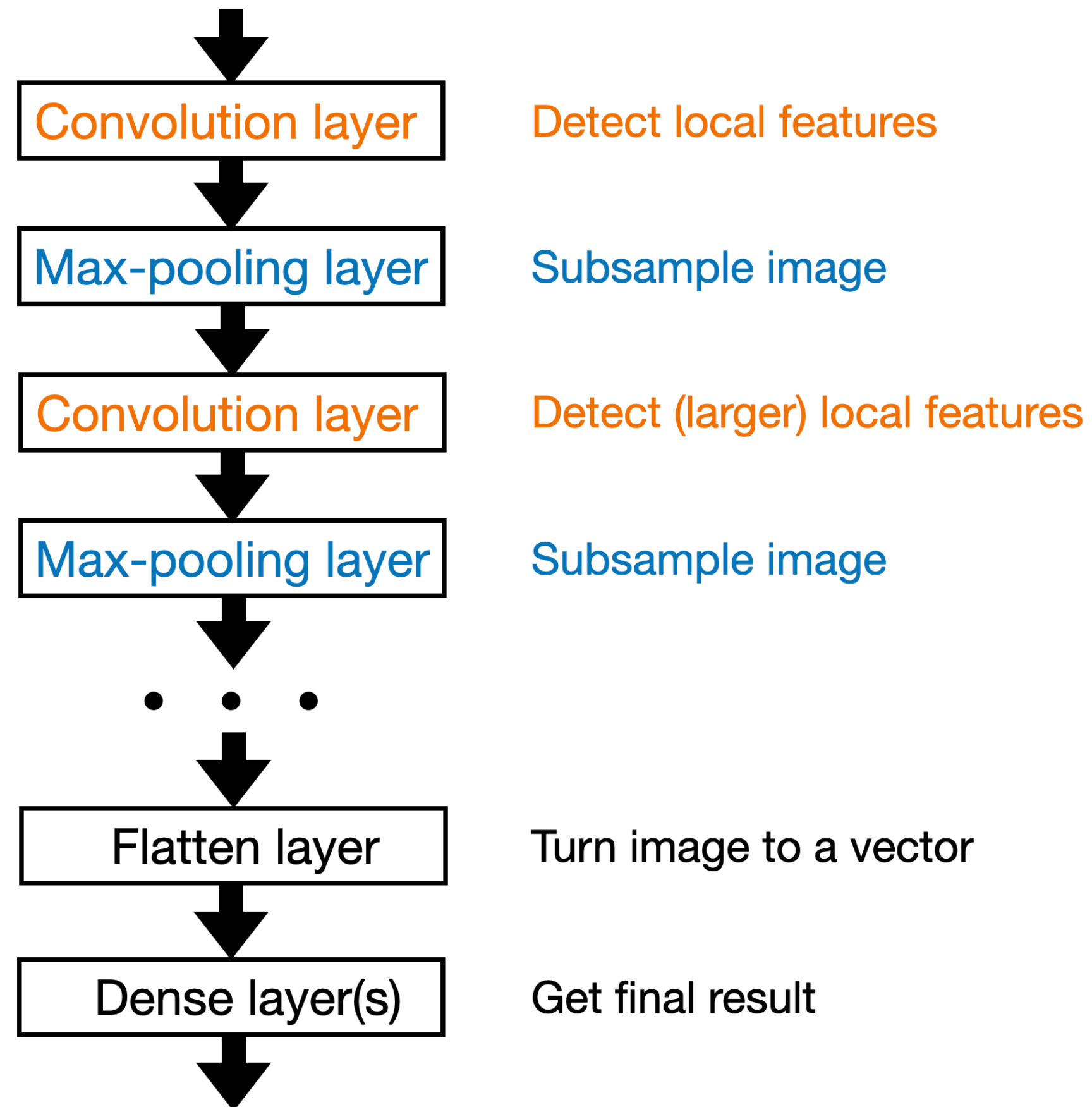
3. Small local patterns are combined to become a larger local pattern.

CNN for Images (or image-like data)



4. Subsampling an image can still enable us to detect a relatively large pattern.

A common CNN Architecture



Quiz questions:

1. What properties of images inspired the CNN architecture?
2. What is the basic architecture of CNN?

Roadmap of this lecture:

1. How a convolutional neural network (CNN) works

1.1 CNN architecture

1.2 How a convolution layer works

1.3 How max-pooling and flatten layers work

2. Example of CNN for MNIST



Roadmap of this lecture (continued):

3. Train a CNN without regularization

4. Train a CNN with data augmentation

5. Use feature extraction from a trained model

6. Fine-tune a pre-trained model



How a convolution layer works

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1

Filter 1

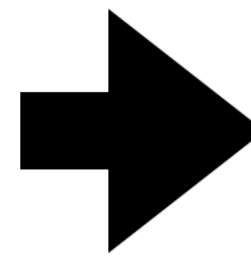
-1	-1	-1
1	1	1
-1	-1	-1

Filter 2

-1	-1	1
-1	1	-1
1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1



Feature map for filter 1

Filter 1

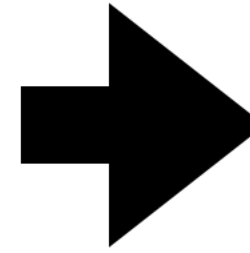
-1	-1	-1
1	1	1
-1	-1	-1

Filter 2

-1	-1	1
-1	1	-1
1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1



Feature map for filter 1

Feature map for filter 2

Filter 1

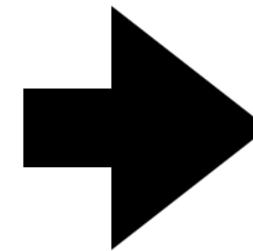
-1	-1	-1
1	1	1
-1	-1	-1

Filter 2

-1	-1	1
-1	1	-1
1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1



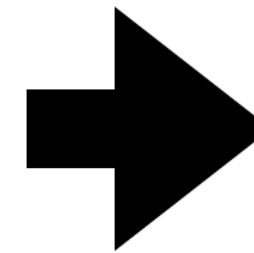
Feature map for filter 1

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1



Feature map for filter 1

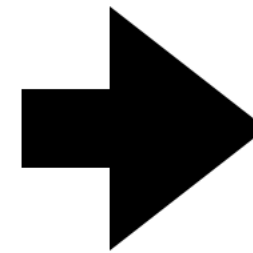
-1			

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1



Feature map for filter 1

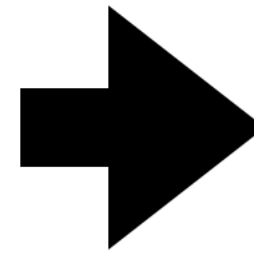
-1	0		

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1



Feature map for filter 1

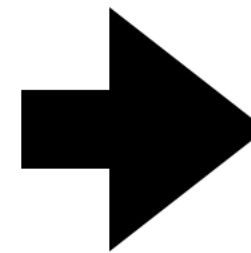
-1	0	2	

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1



Feature map for filter 1

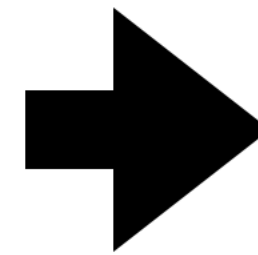
-1	0	2	3

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1



Feature map for filter 1

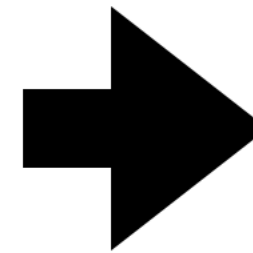
-1	0	2	3
-2			

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1



Feature map for filter 1

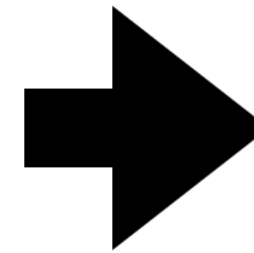
-1	0	2	3
-2	-3		

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1



Feature map for filter 1

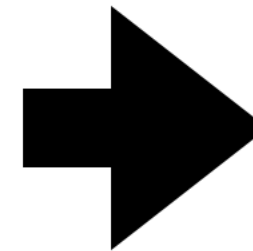
-1	0	2	3
-2	-3	-4	

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1



Feature map for filter 1

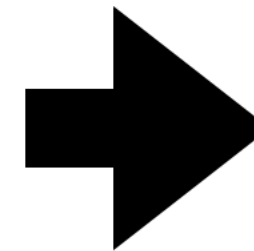
-1	0	2	3
-2	-3	-4	-5

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1



Feature map for filter 1

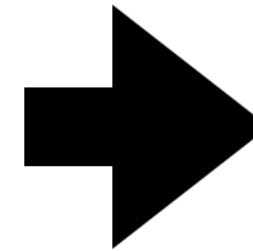
-1	0	2	3
-2	-3	-4	-5
-1			

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1



Feature map for filter 1

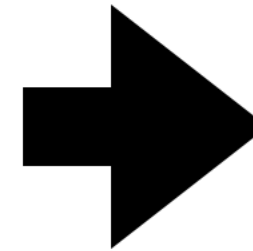
-1	0	2	3
-2	-3	-4	-5
-1	0		

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1



Feature map for filter 1

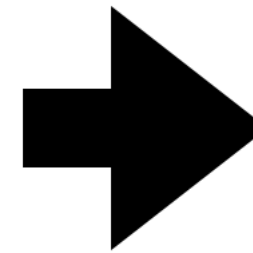
-1	0	2	3
-2	-3	-4	-5
-1	0	1	

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1



Feature map for filter 1

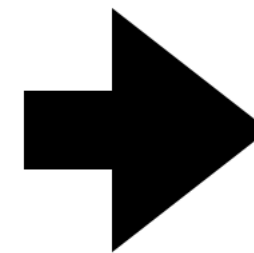
-1	0	2	3
-2	-3	-4	-5
-1	0	1	1

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1



Feature map for filter 1

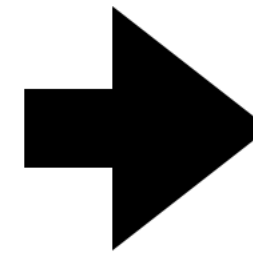
-1	0	2	3
-2	-3	-4	-5
-1	0	1	1
-1			

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1



Feature map for filter 1

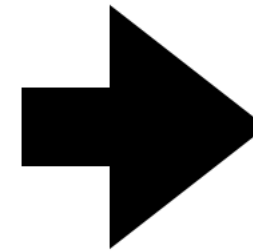
-1	0	2	3
-2	-3	-4	-5
-1	0	1	1
-1	-2		

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1



Feature map for filter 1

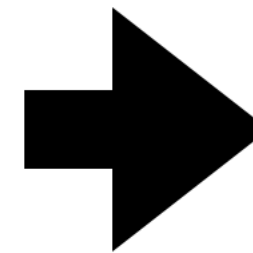
-1	0	2	3
-2	-3	-4	-5
-1	0	1	1
-1	-2	-2	

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1



Feature map for filter 1

-1	0	2	3
-2	-3	-4	-5
-1	0	1	1
-1	-2	-2	-3

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1

Feature map for filter 1

-1	0	2	3
-2	-3	-4	-5
-1	0	1	1
-1	-2	-2	-3

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Filter 2

-1	-1	1
-1	1	-1
1	-1	-1

Feature map for filter 2

-3			

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1

Feature map for filter 1

-1	0	2	3
-2	-3	-4	-5
-1	0	1	1
-1	-2	-2	-3

Feature map for filter 2

-3	-2		

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Filter 2

-1	-1	1
-1	1	-1
1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1

Feature map for filter 1

-1	0	2	3
-2	-3	-4	-5
-1	0	1	1
-1	-2	-2	-3

Feature map for filter 2

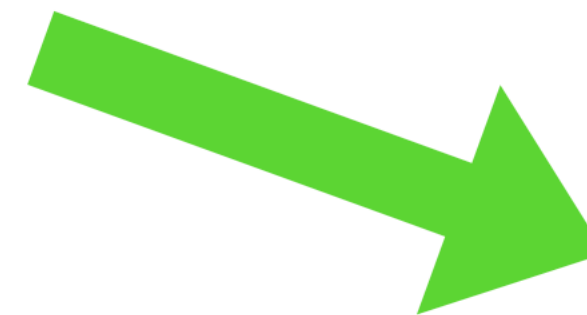
-3	-2	0	

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Filter 2

-1	-1	1
-1	1	-1
1	-1	-1



Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1

Feature map for filter 1

-1	0	2	3
-2	-3	-4	-5
-1	0	1	1
-1	-2	-2	-3

Feature map for filter 2

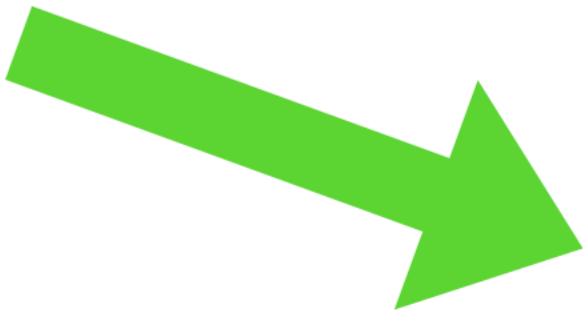
-3	-2	0	-1

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Filter 2

-1	-1	1
-1	1	-1
1	-1	-1



Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1

Feature map for filter 1

-1	0	2	3
-2	-3	-4	-5
-1	0	1	1
-1	-2	-2	-3

Feature map for filter 2

-3	-2	0	-1
-2			

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Filter 2

-1	-1	1
-1	1	-1
1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1

Feature map for filter 1

-1	0	2	3
-2	-3	-4	-5
-1	0	1	1
-1	-2	-2	-3

Feature map for filter 2

-3	-2	0	-1
-2	-3		

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Filter 2

-1	-1	1
-1	1	-1
1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1

Feature map for filter 1

-1	0	2	3
-2	-3	-4	-5
-1	0	1	1
-1	-2	-2	-3

Feature map for filter 2

-3	-2	0	-1
-2	-3	0	

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Filter 2

-1	-1	1
-1	1	-1
1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1

Feature map for filter 1

-1	0	2	3
-2	-3	-4	-5
-1	0	1	1
-1	-2	-2	-3

Feature map for filter 2

-3	-2	0	-1
-2	-3	0	-1

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Filter 2

-1	-1	1
-1	1	-1
1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1

Feature map for filter 1

-1	0	2	3
-2	-3	-4	-5
-1	0	1	1
-1	-2	-2	-3

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Filter 2

-1	-1	1
-1	1	-1
1	-1	-1

Feature map for filter 2

-3	-2	0	-1
-2	-3	0	-1
-3			

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1

Feature map for filter 1

-1	0	2	3
-2	-3	-4	-5
-1	0	1	1
-1	-2	-2	-3

Feature map for filter 2

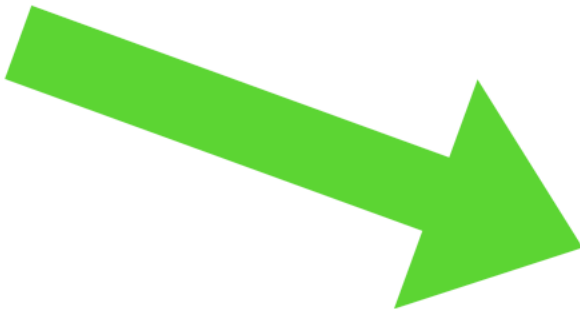
-3	-2	0	-1
-2	-3	0	-1
-3	0		

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Filter 2

-1	-1	1
-1	1	-1
1	-1	-1



Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1

Feature map for filter 1

-1	0	2	3
-2	-3	-4	-5
-1	0	1	1
-1	-2	-2	-3

Feature map for filter 2

-3	-2	0	-1
-2	-3	0	-1
-3	0	-1	

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Filter 2

-1	-1	1
-1	1	-1
1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1

Feature map for filter 1

-1	0	2	3
-2	-3	-4	-5
-1	0	1	1
-1	-2	-2	-3

Feature map for filter 2

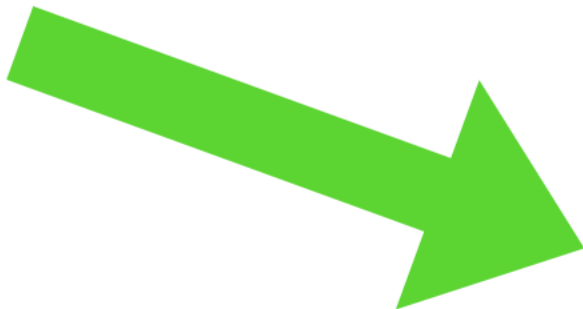
-3	-2	0	-1
-2	-3	0	-1
-3	0	-1	-3

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Filter 2

-1	-1	1
-1	1	-1
1	-1	-1



Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1

Feature map for filter 1

-1	0	2	3
-2	-3	-4	-5
-1	0	1	1
-1	-2	-2	-3

Feature map for filter 2

-3	-2	0	-1
-2	-3	0	-1
-3	0	-1	-3
3			

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Filter 2

-1	-1	1
-1	1	-1
1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1

Feature map for filter 1

-1	0	2	3
-2	-3	-4	-5
-1	0	1	1
-1	-2	-2	-3

Feature map for filter 2

-3	-2	0	-1
-2	-3	0	-1
-3	0	-1	-3
3	-2		

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Filter 2

-1	-1	1
-1	1	-1
1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1

Feature map for filter 1

-1	0	2	3
-2	-3	-4	-5
-1	0	1	1
-1	-2	-2	-3

Feature map for filter 2

-3	-2	0	-1
-2	-3	0	-1
-3	0	-1	-3
3	-2	-4	

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Filter 2

-1	-1	1
-1	1	-1
1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1

Feature map for filter 1

-1	0	2	3
-2	-3	-4	-5
-1	0	1	1
-1	-2	-2	-3

Feature map for filter 2

-3	-2	0	-1
-2	-3	0	-1
-3	0	-1	-3
3	-2	-4	1

Filter 1

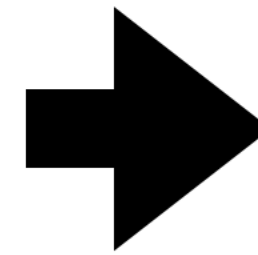
-1	-1	-1
1	1	1
-1	-1	-1

Filter 2

-1	-1	1
-1	1	-1
1	-1	-1

Image

1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1



Feature map for filter 1

-1	0	2	3	
-2	-3	-2	0	-1
-1	-2	-3	0	-1
-1	-3	0	-1	-3
	3	-2	-4	1

Feature map for filter 2

Filter 1

-1	-1	-1
1	1	1
-1	-1	-1

Filter 2

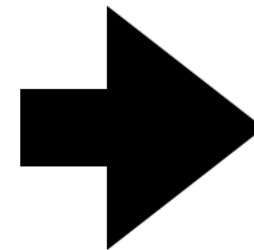
-1	-1	1
-1	1	-1
1	-1	-1

Each filter has $3 \times 3 = 9$ parameters, which are shared by $4 \times 4 = 16$ neurons.

For large images, the saving is big compared to full-connected networks.

Image

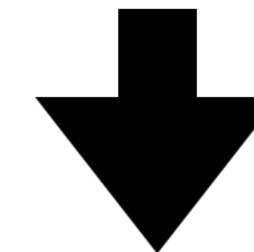
1	1	0	0	0	0
1	1	0	1	1	1
0	1	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	0
1	0	0	1	0	1



Feature map for filter 1

-1	0	2	3	
-2	-3	-2	0	-1
-1	-2	-3	0	-1
-1	-3	0	-1	-3
-1	3	-2	-4	1

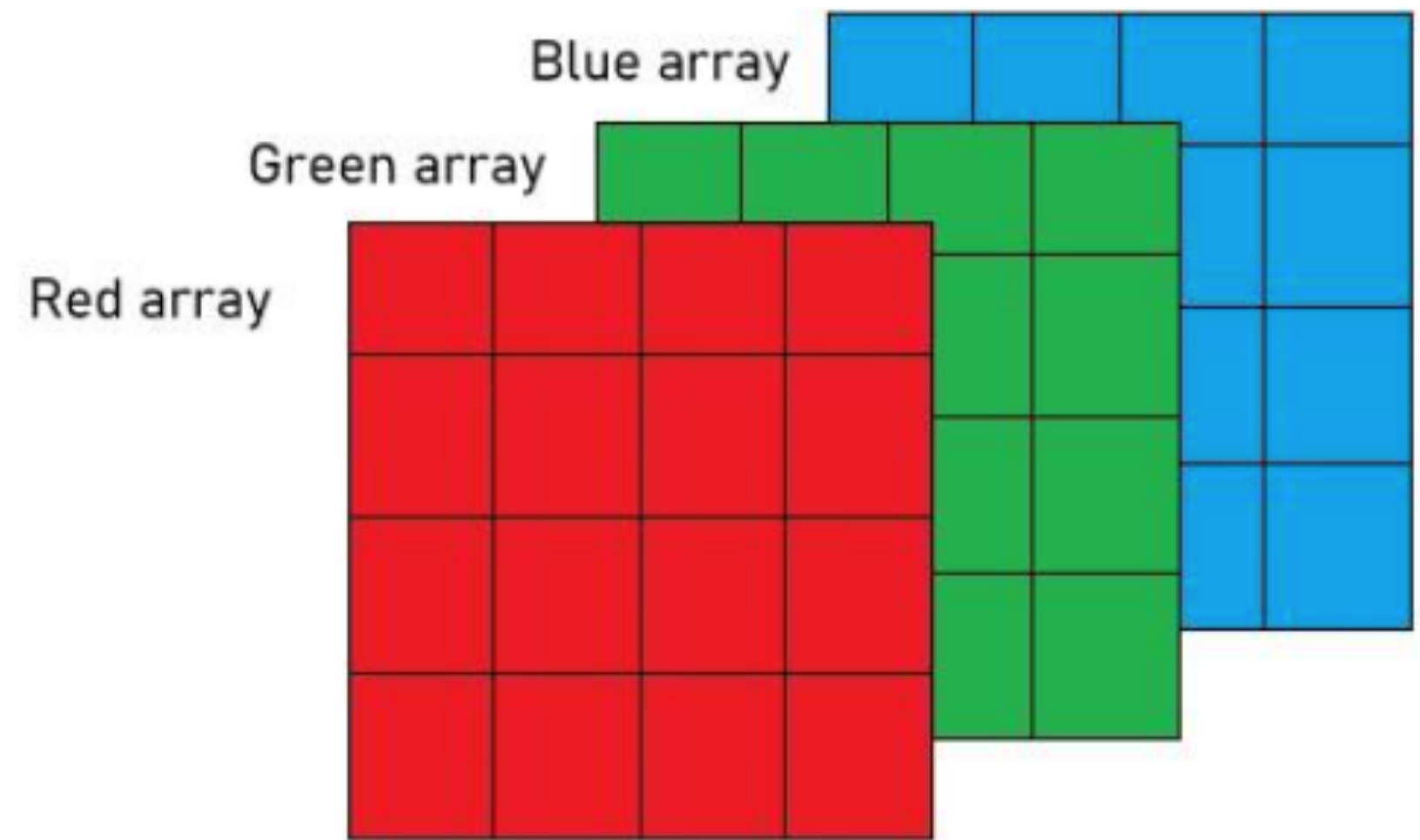
Feature map for filter 2



An image of depth 2, also called “an image of 2 channels”.

Number of channels equals the number of convolution filters.

A colored image is an image of 3 channels

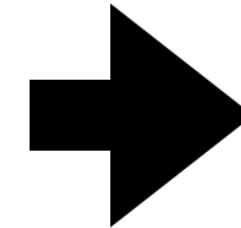


Depth of a convolution filter = # of channels in the image

Image

0	0	0	1	0	1	
1	0	0	1	0	1	0
1	1	1	0	0	0	0
C	1	1	0	1	1	1
C	1	0	1	0	0	0
C	0	0	1	1	0	1
C	0	1	0	0	1	0
	0	1	0	0	1	0
	1	0	0	1	0	1

Feature map



Convolution Filter

-1	-1	-1		
1	-1	-1	1	
	-1	1	1	-1
	-1	-1	-1	-1
	1	-1	-1	1

Quiz questions:

1. How does convolution work in a convolution layer?
2. Why does a convolution layer have fewer parameters to train than a fully-connected layer?
3. What is the relation between the number of channels in an image (or feature map) and the depth of a convolution filter?

Roadmap of this lecture:

1. How a convolutional neural network (CNN) works

1.1 CNN architecture

1.2 How a convolution layer works

1.3 How max-pooling and flatten layers work

2. Example of CNN for MNIST



Roadmap of this lecture (continued):

3. Train a CNN without regularization

4. Train a CNN with data augmentation

5. Use feature extraction from a trained model

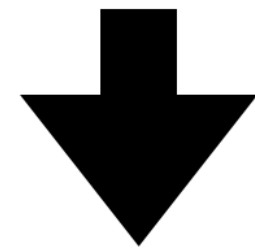
6. Fine-tune a pre-trained model



How a max-pooling layer works

Feature map for filter 1

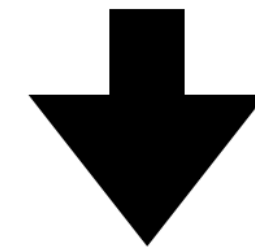
-1	0	2	3
-2	-3	-4	-5
-1	0	1	1
-1	-2	-2	-3



Subsampled
feature map
for filter 1

Feature map for filter 2

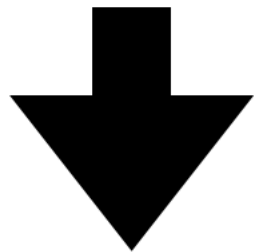
-3	-2	0	-1
-2	-3	0	-1
-3	0	-1	-3
3	-2	-4	1



Subsampled
feature map
for filter 2

Feature map for filter 1

-1	0	2	3
-2	-3	-4	-5
-1	0	1	1
-1	-2	-2	-3

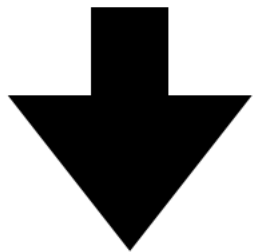


0	3
0	1

Subsampled
feature map
for filter 1

Feature map for filter 2

-3	-2	0	-1
-2	-3	0	-1
-3	0	-1	-3
3	-2	-4	1



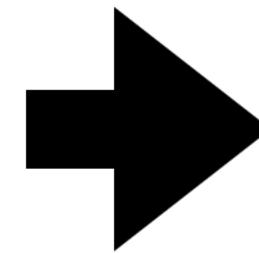
-2	0
3	1

Subsampled
feature map
for filter 2

Feature map for filter 1

-1	0	2	3	
-2	-3	-2	0	-1
-1	-2	-3	0	-1
-1	-3	0	-1	-3
-1	3	-2	-4	1

Feature map for filter 2



Subsampled feature map for filter 1

0	3
-2	0
3	1

Subsampled feature map for filter 2

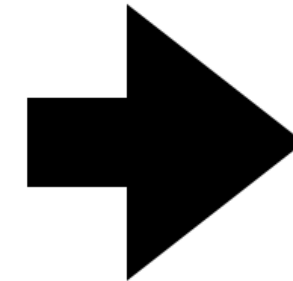
The “image” becomes smaller after max-pooling, but the number of channels is unchanged.

How a flatten layer works

Subsampled
feature map
for filter 1

0	3
1	-2 0
	3 1

Subsampled
feature map
for filter 2



0
3
0
1
-2
0
3
1

Quiz questions:

1. How does a max-pooling layer work?
2. How does a flatten layer work?

Roadmap of this lecture:

1. How a convolutional neural network (CNN) works

1.1 CNN architecture

1.2 How a convolution layer works

1.3 How max-pooling and flatten layers work

2. Example of CNN for MNIST



Roadmap of this lecture (continued):

3. Train a CNN without regularization

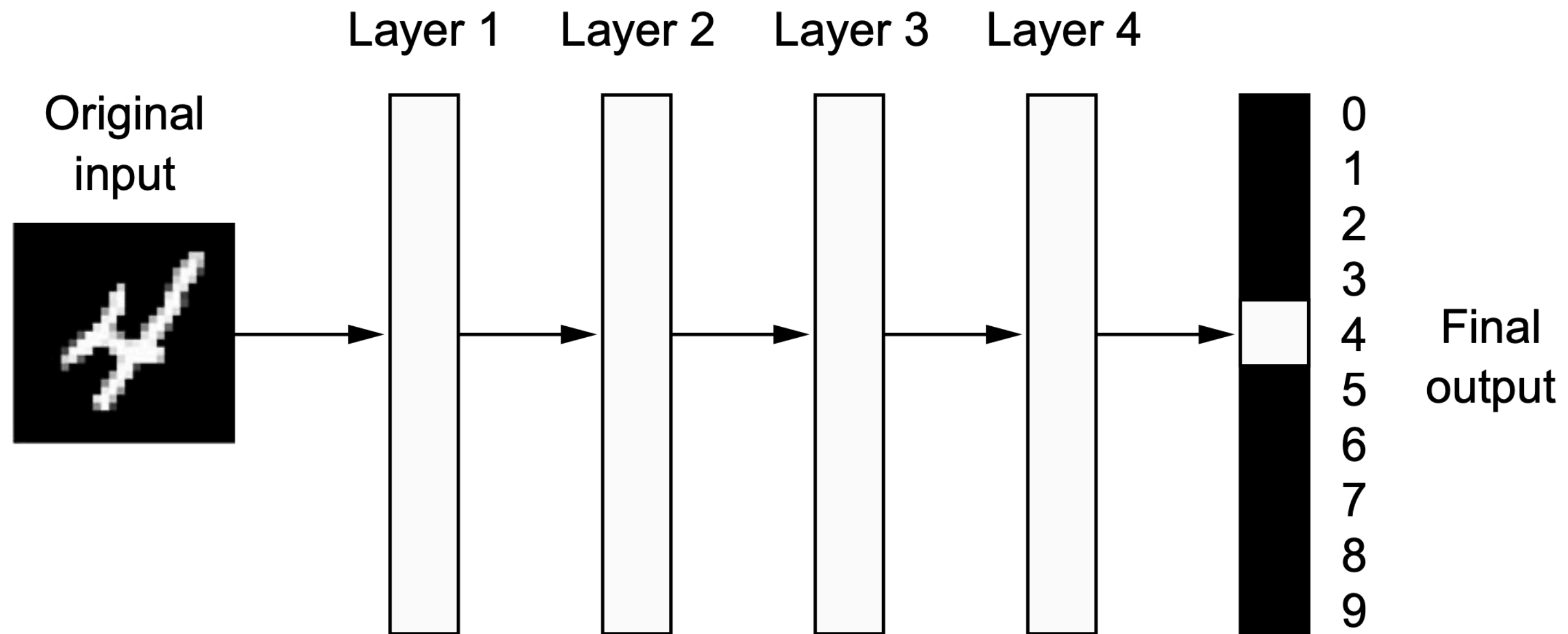
4. Train a CNN with data augmentation

5. Use feature extraction from a trained model

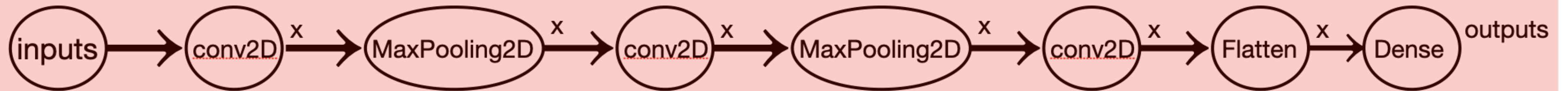
6. Fine-tune a pre-trained model



Example of **CNN**: for MNIST



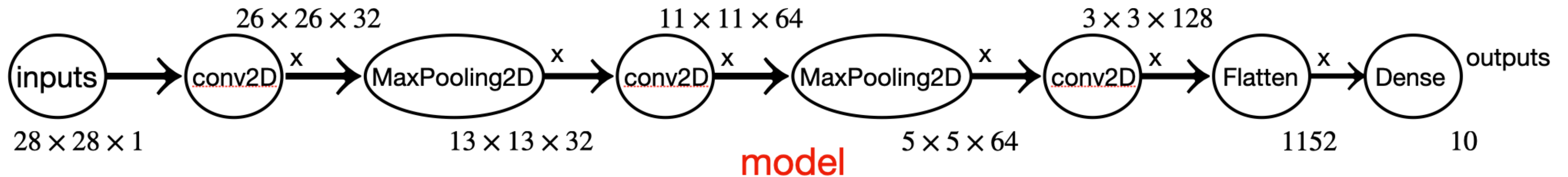
```
from tensorflow import keras
from tensorflow.keras import layers
inputs = keras.Input(shape=(28, 28, 1))
x = layers.Conv2D(filters=32, kernel_size=3, activation="relu")(inputs)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=64, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=128, kernel_size=3, activation="relu")(x)
x = layers.Flatten()(x)
outputs = layers.Dense(10, activation="softmax")(x)
model = keras.Model(inputs=inputs, outputs=outputs)
```



model


```
from tensorflow.keras.datasets import mnist
```

```
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
train_images = train_images.reshape((60000, 28, 28, 1))
train_images = train_images.astype("float32") / 255
test_images = test_images.reshape((10000, 28, 28, 1))
test_images = test_images.astype("float32") / 255
model.compile(optimizer="rmsprop",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
model.fit(train_images, train_labels, epochs=5, batch_size=64)
```



Listing 8.4 Evaluating the convnet

```
>>> test_loss, test_acc = model.evaluate(test_images, test_labels)
>>> print(f"Test accuracy: {test_acc:.3f}")
Test accuracy: 0.991
```

Good performance

Understanding **border effects** and padding

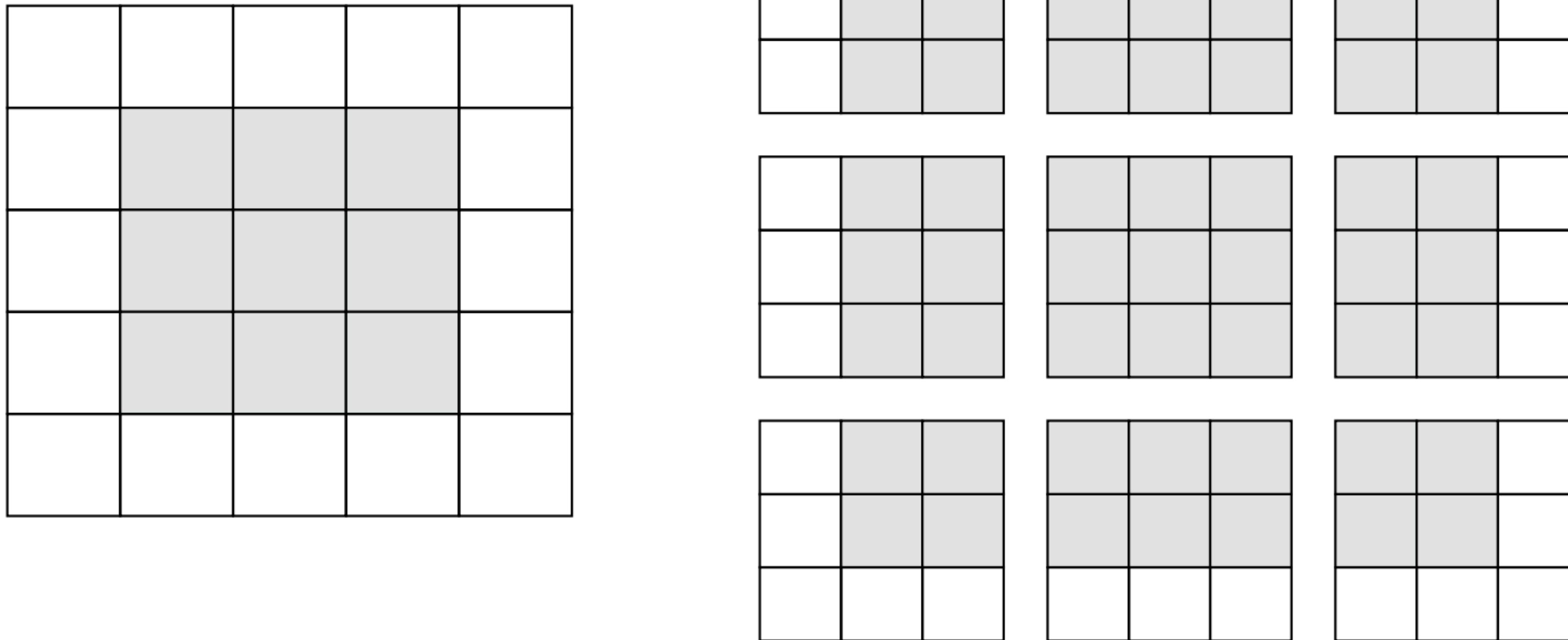


Figure 8.5 Valid locations of 3 × 3 patches in a 5 × 5 input feature map

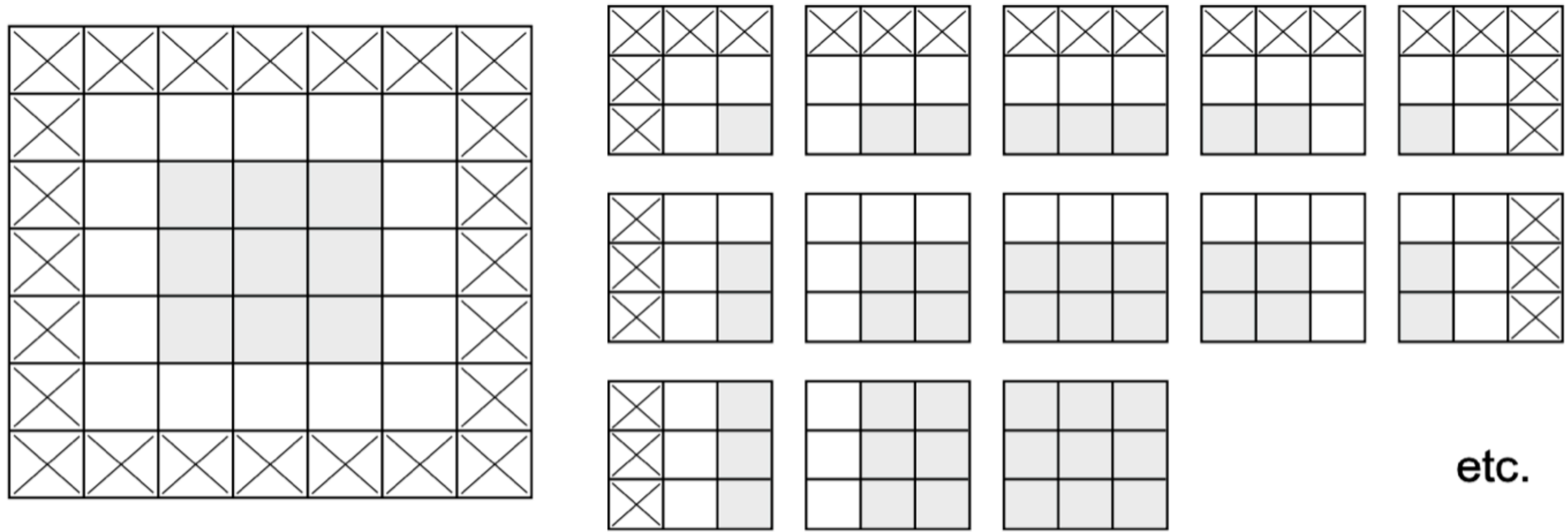


Figure 8.6 Padding a 5×5 input in order to be able to extract 25 3×3 patches

In Conv2D layers, padding is configurable via the padding argument, which takes two values: "valid", which means no padding (only valid window locations will be used), and "same", which means "pad in such a way as to have an output with the same width and height as the input." The padding argument defaults to "valid".

Understanding **convolution strides**

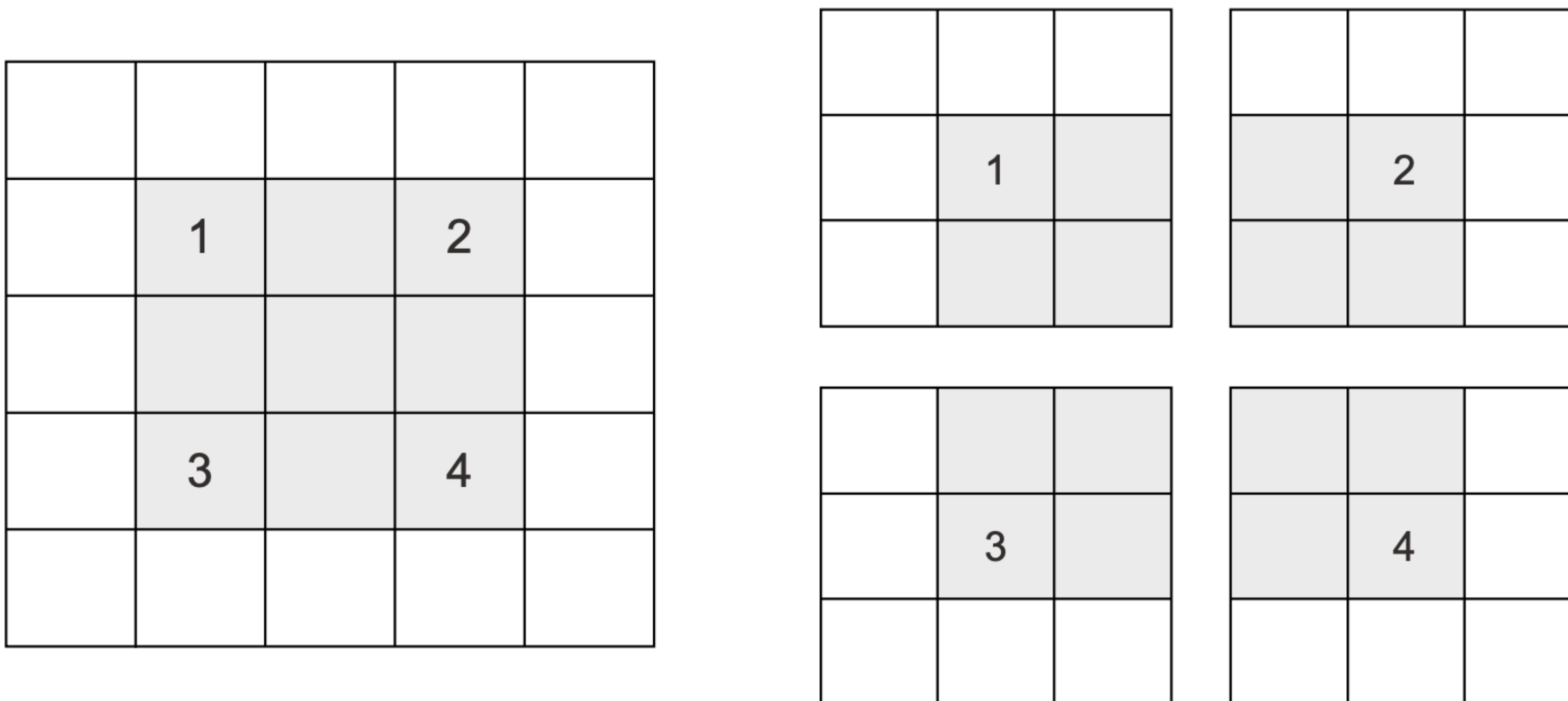


Figure 8.7 3×3 convolution patches with 2×2 strides

Quiz questions:

1. How to design a CNN using Keras?
2. What is the border effect for a convolution layer?
3. What is stride for convolution?

Roadmap of this lecture:

1. How a convolutional neural network (CNN) works

1.1 CNN architecture

1.2 How a convolution layer works

1.3 How max-pooling and flatten layers work

2. Example of CNN for MNIST



Roadmap of this lecture (continued):

3. Train a CNN without regularization

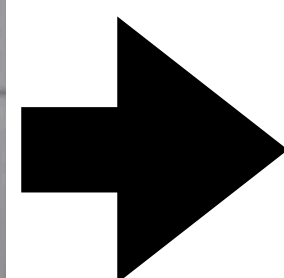
4. Train a CNN with data augmentation

5. Use feature extraction from a trained model

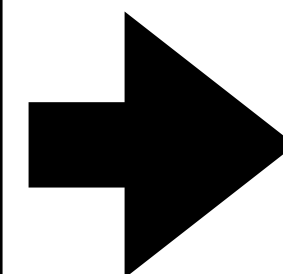
6. Fine-tune a pre-trained model



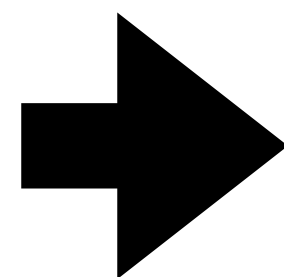
Task: dog-versus-cat image recognition



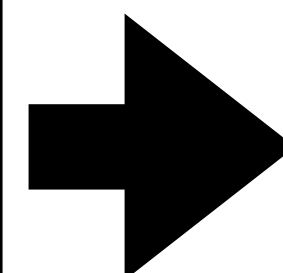
model



cat



model



dog

Download dataset

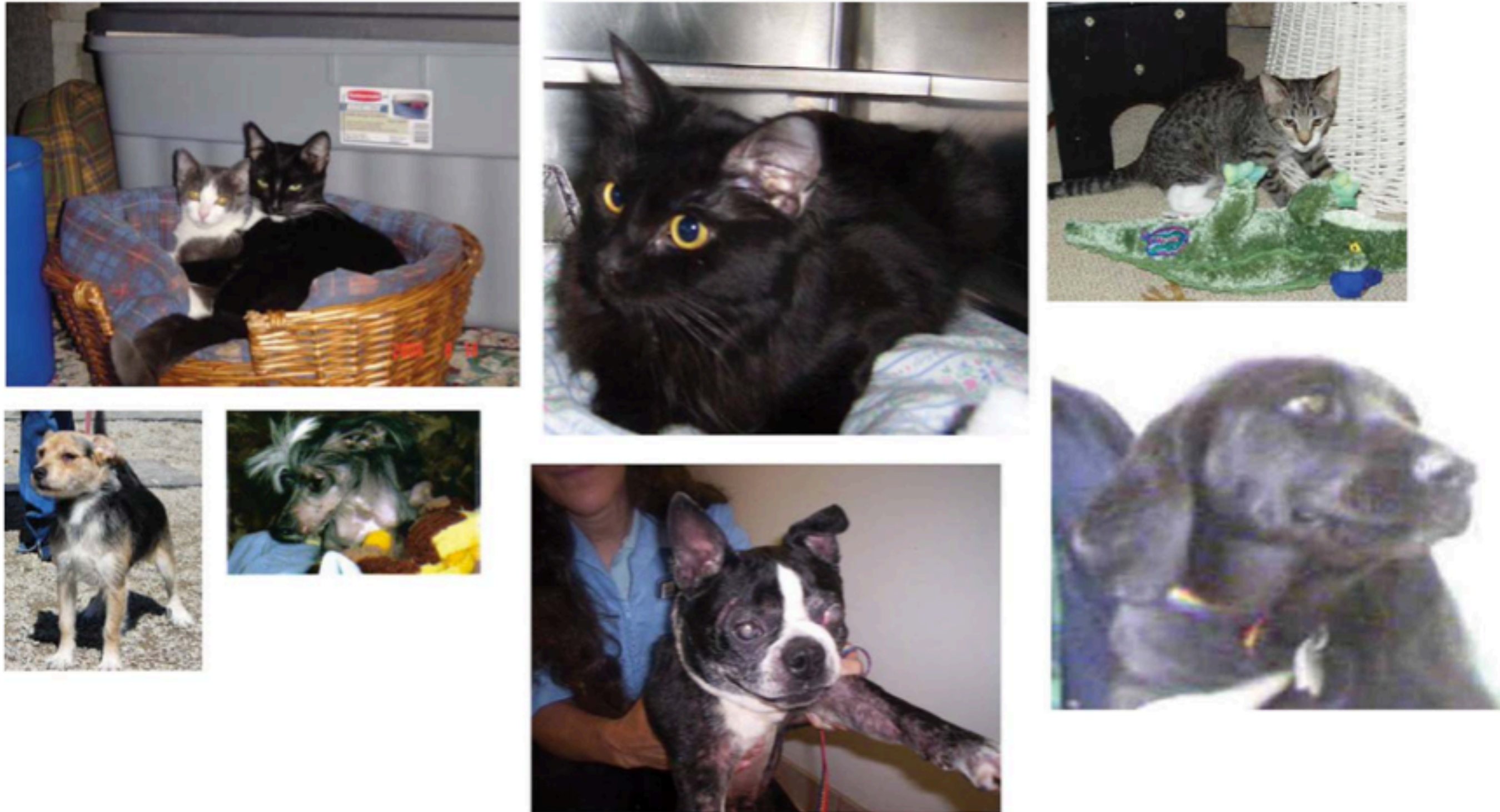


Figure 8.8 Samples from the Dogs vs. Cats dataset. Sizes weren't modified: the samples come in different sizes, colors, backgrounds, etc.

Build the model

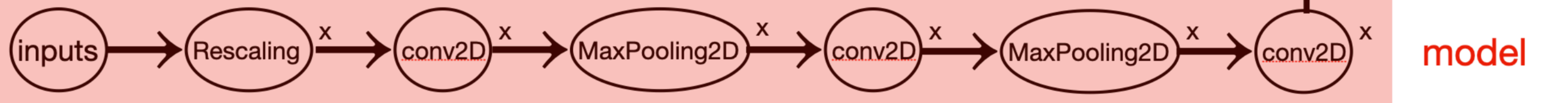
Listing 8.7 Instantiating a small convnet for dogs vs. cats classification

```
from tensorflow import keras
from tensorflow.keras import layers

inputs = keras.Input(shape=(180, 180, 3))
x = layers.Rescaling(1./255)(inputs)
x = layers.Conv2D(filters=32, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=64, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=128, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=256, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=256, kernel_size=3, activation="relu")(x)
x = layers.Flatten()(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs=inputs, outputs=outputs)
```

The model
expects
RGB images
of size
 180×180 .

Rescale
inputs to the
[0, 1] range
by dividing
them by 255.

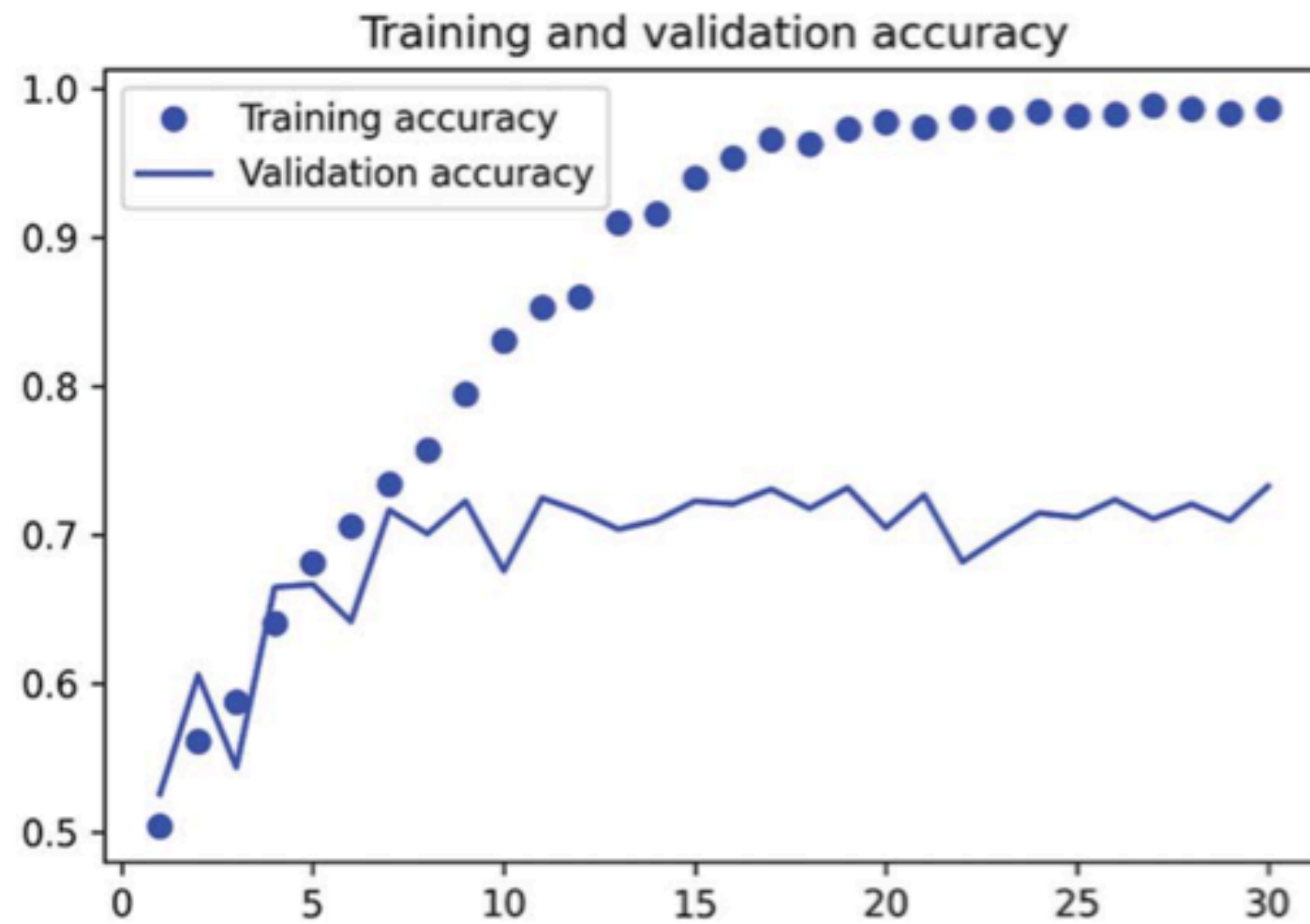


```
model.compile(loss="binary_crossentropy",
              optimizer="rmsprop",
              metrics=["accuracy"])
```

Listing 8.11 Fitting the model using a Dataset

```
callbacks = [
    keras.callbacks.ModelCheckpoint(
        filepath="convnet_from_scratch.keras",
        save_best_only=True,
        monitor="val_loss")
]

history = model.fit(
    train_dataset,
    epochs=30,
    validation_data=validation_dataset,
    callbacks=callbacks)
```



```
test_model = keras.models.load_model("convnet_from_scratch.keras")
test_loss, test_acc = test_model.evaluate(test_dataset)
print(f"Test accuracy: {test_acc:.3f}")
```

We get a test accuracy of 69.5%.

Quiz question:

1. How to prepare data for a general image dataset?

Roadmap of this lecture:

1. How a convolutional neural network (CNN) works

1.1 CNN architecture

1.2 How a convolution layer works

1.3 How max-pooling and flatten layers work

2. Example of CNN for MNIST



Roadmap of this lecture (continued):

3. Train a CNN without regularization

4. Train a CNN with data augmentation

5. Use feature extraction from a trained model

6. Fine-tune a pre-trained model

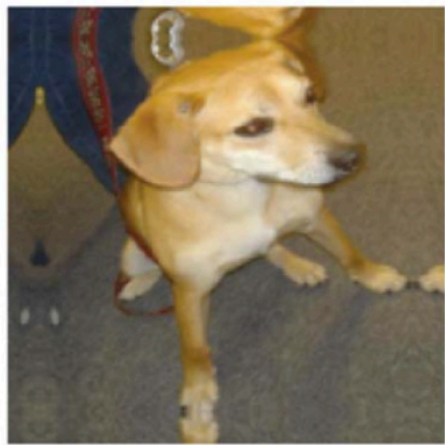
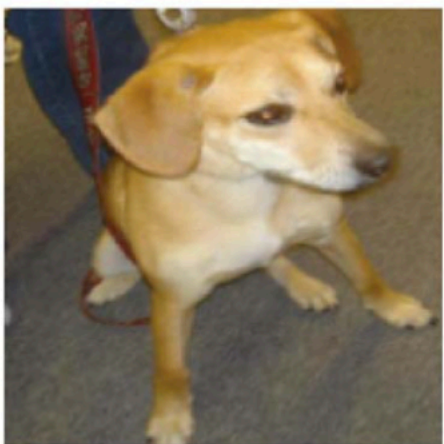
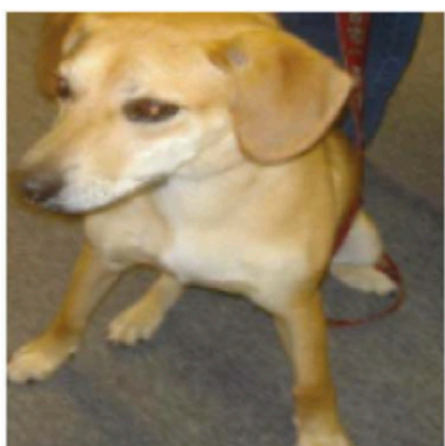


Use **data augmentation**

Data augmentation can be done by adding a number of **data augmentation layers** at the start of the model.

```
data_augmentation = keras.Sequential(  
    [  
        layers.RandomFlip("horizontal"),  
        layers.RandomRotation(0.1),  
        layers.RandomZoom(0.2),  
    ]  
)
```


- `RandomFlip("horizontal")`—Applies horizontal flipping to a random 50% of the images that go through it
- `RandomRotation(0.1)`—Rotates the input images by a random value in the range $[-10\%, +10\%]$ (these are fractions of a full circle—in degrees, the range would be $[-36 \text{ degrees}, +36 \text{ degrees}]$)
- `RandomZoom(0.2)`—Zooms in or out of the image by a random factor in the range $[-20\%, +20\%]$



Note: data augmentation is inactive during inference.

Note: “Dropout” is also inactive during inference.

Fight overfitting

```
inputs = keras.Input(shape=(180, 180, 3))
x = data_augmentation(inputs)
x = layers.Rescaling(1./255)(x)
x = layers.Conv2D(filters=32, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=64, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=128, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=256, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=256, kernel_size=3, activation="relu")(x)
x = layers.Flatten()(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs=inputs, outputs=outputs)

model.compile(loss="binary_crossentropy",
              optimizer="rmsprop",
              metrics=["accuracy"])
```

Train the regularized model

```
callbacks = [  
    keras.callbacks.ModelCheckpoint(  
        filepath="convnet_from_scratch_with_augmentation.keras",  
        save_best_only=True,  
        monitor="val_loss")  
]  
history = model.fit(  
    train_dataset,  
    epochs=100,  
    validation_data=validation_dataset,  
    callbacks=callbacks)
```

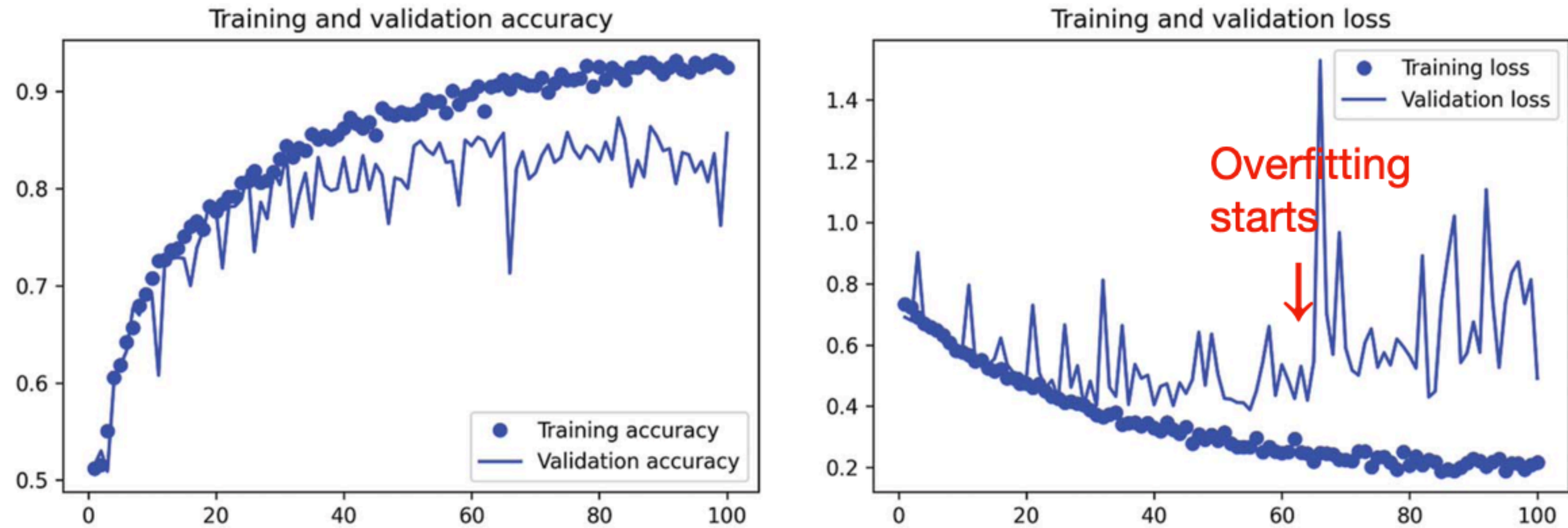


Figure 8.11 Training and validation metrics with data augmentation

```
test_model = keras.models.load_model(
    "convnet_from_scratch_with_augmentation.keras")
test_loss, test_acc = test_model.evaluate(test_dataset)
print(f"Test accuracy: {test_acc:.3f}")
```

We get a test accuracy of 83.5%. It's starting to look good!

Quiz questions:

1. When is data augmentation needed?
2. How to use data augmentation for a CNN?

Roadmap of this lecture:

1. How a convolutional neural network (CNN) works

1.1 CNN architecture

1.2 How a convolution layer works

1.3 How max-pooling and flatten layers work

2. Example of CNN for MNIST



Roadmap of this lecture (continued):

3. Train a CNN without regularization

4. Train a CNN with data augmentation

5. Use feature extraction from a trained model

6. Fine-tune a pre-trained model



Leveraging a **pre-trained model**

Use a good (and usually large) model trained on a large (and related) dataset.

Such a pre-trained model can extract good features from input data.

Two ways to use a pre-trained model:

- 1) feature extraction

- 2) fine-tuning

Feature extraction with a pre-trained model

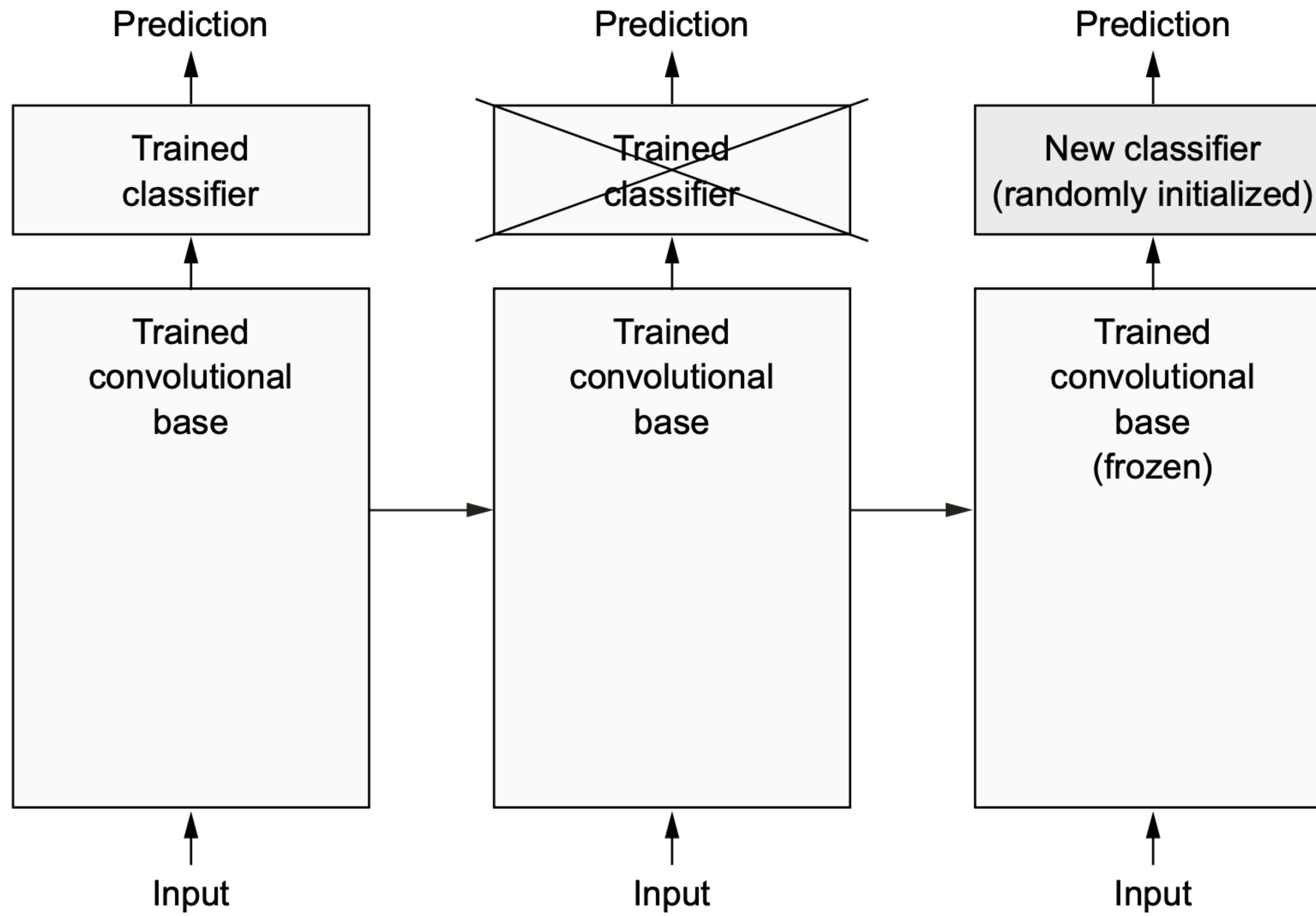


Figure 8.12 Swapping classifiers while keeping the same convolutional base

Get **convolutional base** from a pre-trained VGG16 model

In Keras, we **freeze** a layer or model by setting its trainable attribute to False.

Listing 8.23 Instantiating and freezing the VGG16 convolutional base

```
conv_base = keras.applications.vgg16.VGG16(  
    weights="imagenet",  
    include_top=False)  
conv_base.trainable = False
```



```
>>> conv_base.summary()
```

```
Model: "vgg16"
```

Layer (type)	Output Shape	Param #
=====		
input_19 (InputLayer)	[(None, 180, 180, 3)]	0
block1_conv1 (Conv2D)	(None, 180, 180, 64)	1792
block1_conv2 (Conv2D)	(None, 180, 180, 64)	36928
block1_pool (MaxPooling2D)	(None, 90, 90, 64)	0
block2_conv1 (Conv2D)	(None, 90, 90, 128)	73856

block2_conv2 (Conv2D)	(None, 90, 90, 128)	147584
block2_pool (MaxPooling2D)	(None, 45, 45, 128)	0
block3_conv1 (Conv2D)	(None, 45, 45, 256)	295168
block3_conv2 (Conv2D)	(None, 45, 45, 256)	590080
block3_conv3 (Conv2D)	(None, 45, 45, 256)	590080
block3_pool (MaxPooling2D)	(None, 22, 22, 256)	0
block4_conv1 (Conv2D)	(None, 22, 22, 512)	1180160
block4_conv2 (Conv2D)	(None, 22, 22, 512)	2359808
block4_conv3 (Conv2D)	(None, 22, 22, 512)	2359808
block4_pool (MaxPooling2D)	(None, 11, 11, 512)	0
block5_conv1 (Conv2D)	(None, 11, 11, 512)	2359808
block5_conv2 (Conv2D)	(None, 11, 11, 512)	2359808
block5_conv3 (Conv2D)	(None, 11, 11, 512)	2359808
block5_pool (MaxPooling2D)	(None, 5, 5, 512)	0
=====		
Total params: 14,714,688		
Trainable params: 14,714,688		
Non-trainable params: 0		

Feature extraction together with data augmentation

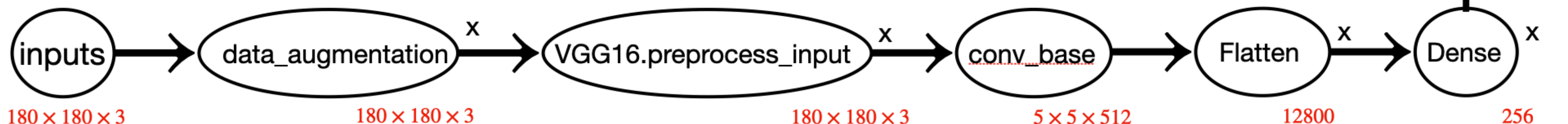
Listing 8.25 Adding a data augmentation stage and a classifier to the convolutional base

```
data_augmentation = keras.Sequential(  
    [  
        layers.RandomFlip("horizontal"),  
        layers.RandomRotation(0.1),  
        layers.RandomZoom(0.2),  
    ]  
)
```

```
inputs = keras.Input(shape=(180, 180, 3))  
x = data_augmentation(inputs)  
x = keras.applications.vgg16.preprocess_input(x)  
x = conv_base(x)  
x = layers.Flatten()(x)  
x = layers.Dense(256)(x)  
x = layers.Dropout(0.5)(x)  
outputs = layers.Dense(1, activation="sigmoid")(x)  
model = keras.Model(inputs, outputs)  
model.compile(loss="binary_crossentropy",  
              optimizer="rmsprop",  
              metrics=["accuracy"])
```

Apply data
augmentation.

Apply input
value scaling.



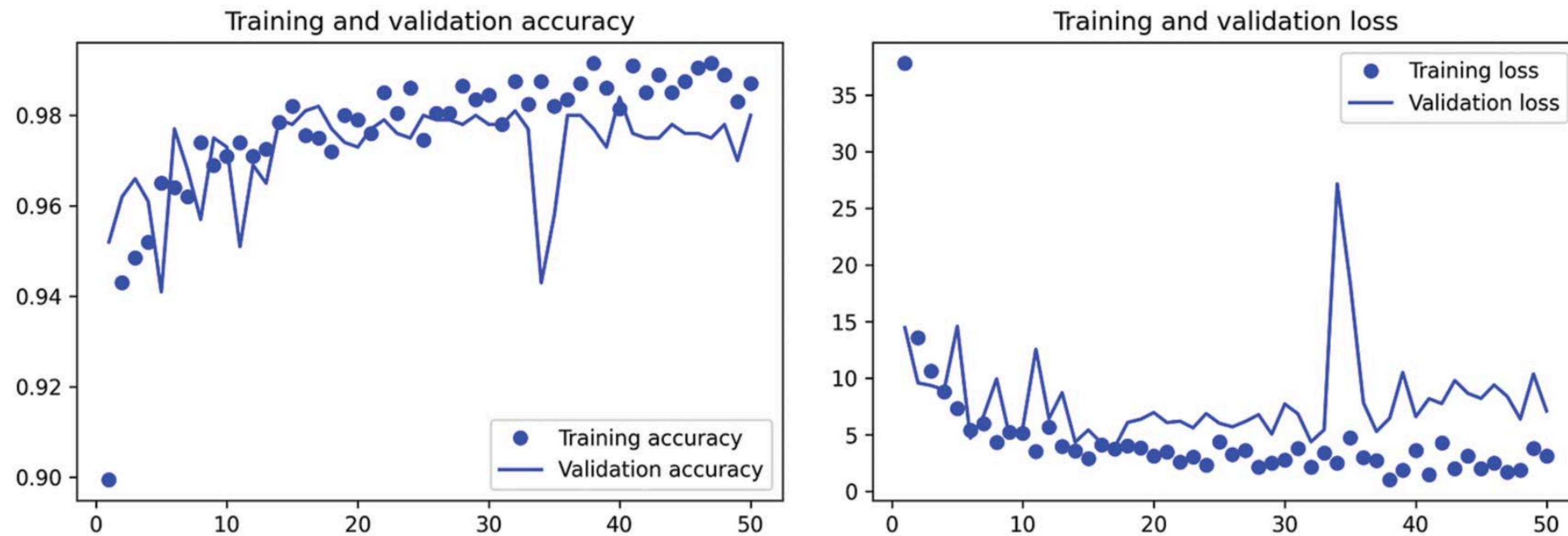


Figure 8.14 Training and validation metrics for feature extraction with data augmentation

```
test_model = keras.models.load_model(
    "feature_extraction_with_data_augmentation.keras")
test_loss, test_acc = test_model.evaluate(test_dataset)
print(f"Test accuracy: {test_acc:.3f}")
```

We get a test accuracy of 97.5%.

Quiz questions:

1. How to use a pretrained model?
2. When can feature extraction from a pretrained model be useful?

Roadmap of this lecture:

1. How a convolutional neural network (CNN) works

1.1 CNN architecture

1.2 How a convolution layer works

1.3 How max-pooling and flatten layers work

2. Example of CNN for MNIST



Roadmap of this lecture (continued):

3. Train a CNN without regularization

4. Train a CNN with data augmentation

5. Use feature extraction from a trained model

6. Fine-tune a pre-trained model



Fine-tuning a pre-trained model

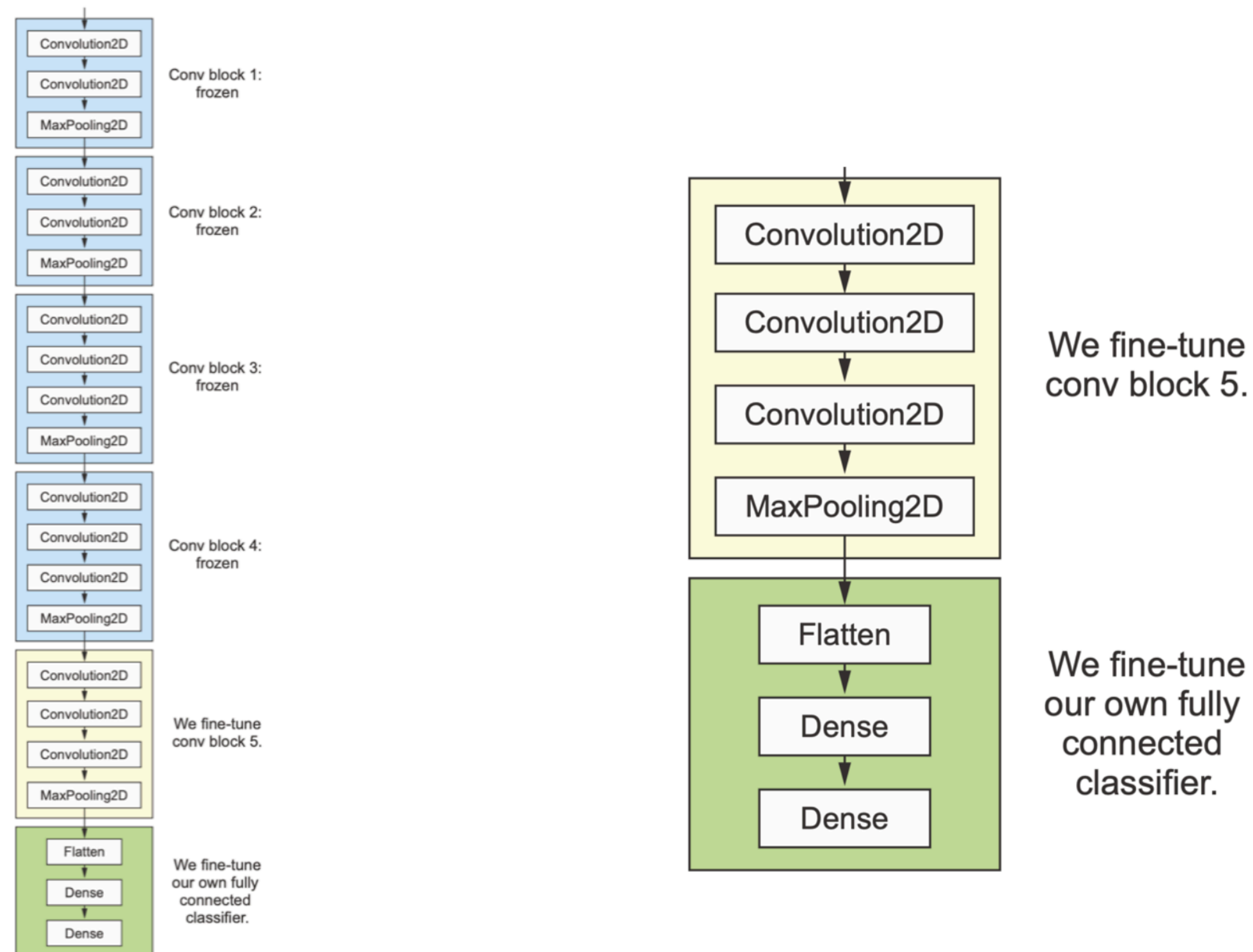


Figure 8.15 Fine-tuning the last convolutional block of the VGG16 network

the steps for fine-tuning a network are as follows:

- 1 Add our custom network on top of an already-trained base network.
- 2 Freeze the base network.
- 3 Train the part we added.
- 4 Unfreeze some layers in the base network.
(Note that you should not unfreeze “batch normalization” layers, which are not relevant here since there are no such layers in VGG16. Batch normalization and its impact on fine-tuning is explained in the next chapter.)
- 5 Jointly train both these layers and the part we added.

You already completed the first three steps when doing feature extraction. Let's proceed with step 4: we'll unfreeze our `conv_base` and then freeze individual layers inside it.

Listing 8.27 Freezing all layers until the fourth from the last

```
conv_base.trainable = True
for layer in conv_base.layers[:-4]:
    layer.trainable = False
```

Now we can begin fine-tuning the model. We'll do this with the RMSprop optimizer, using a very low learning rate. The reason for using a low learning rate is that we want to limit the magnitude of the modifications we make to the representations of the three layers we're fine-tuning. Updates that are too large may harm these representations.

Listing 8.28 Fine-tuning the model

```
model.compile(loss="binary_crossentropy",
              optimizer=keras.optimizers.RMSprop(learning_rate=1e-5),
              metrics=["accuracy"])

callbacks = [
    keras.callbacks.ModelCheckpoint(
        filepath="fine_tuning.keras",
        save_best_only=True,
        monitor="val_loss")
]
history = model.fit(
    train_dataset,
    epochs=30,
    validation_data=validation_dataset,
    callbacks=callbacks)
```

We can finally evaluate this model on the test data:

```
model = keras.models.load_model("fine_tuning.keras")  
test_loss, test_acc = model.evaluate(test_dataset)  
print(f"Test accuracy: {test_acc:.3f}")
```

Test accuracy: 98.5%

Quiz questions:

1. What is fine tuning?
2. When is fine tuning most useful?